

Coding assignment: Weighted Moving Calculations

Introduction

This assessment is designed to evaluate your ability to understand and implement mathematical calculations that are to be carried out in a real-time (online) environment. You will implement an online calculator of various statistical properties of incoming data over a moving fixed-width time window.

Your implementation needs to be in either C++ or Java.

Data stream

We consider data which is steamed in updates, where each update consists of a price and a timestamp. The updated price is in effect starting from the timestamp of the update and until the next update. Timestamps are in nanosecond resolution.

Part 1: Calculation of Time-Weighted Moving Average (TWMA) and Time-Weighted Standard Deviation (TWSTD)

Definition:

We consider window frames of exactly 30 seconds that may end at any arbitrary time 't'. We then define the Time-Weighted Moving Average (TWMA) at time 't' as:

$$TWMA_t = (\sum (Price_i * Duration_i)) / 30 \text{ seconds}$$

Where the sum is over prices appearing in the window frame ending at time 't', and where their durations are limited to this time window (i.e., the duration of the last seen price ends at time 't' and the duration of the first price in the window is taken as if this price started at time t-30, regardless of its original starting time). See Diagram 1 at the end of this document for an example of a window frame and the prices/durations it includes.

In a similar manner, we define the Time-Weighted STD for the same window as:

$$TWSTD_t = \sqrt{(\sum (Duration_i * (Price_i - TWMA_t)^2) / 30 \text{ seconds})}$$

Task:

Implement a calculator which consumes incoming timestamped prices and returns the calculated TWMA / TWSTD upon request. Specifically, you need to implement the following methods:

1. AddNewValue (price, t)
2. GetCurrentTWMA (t)
3. GetCurrentTWSTD(t)

The method 'GetCurrentTWMA(t)' (respectively 'GetCurrentTWSTD(t)') should return the TWMA (respectively TWSTD) value at time 't' given all the prices that were inserted up until time 't' using AddNewValue(). Notice that the calls to GetCurrentTWMA/STD may be with arbitrary 't', i.e., they do not necessarily correspond to the times that values are inserted. You may assume that all the calls are monotonic with respect to time, i.e., if 't1' and 't2' are timestamps of two subsequent calls (to any function) then $t2 \geq t1$. You can also assume that calls to GetCurrentTWMA/STD are only allowed after at least 30 seconds from the first AddNewValue call.

Simulation:

See the attached CSV file security1.csv, which contains two columns (timestamp, price) and use it to simulate a real-time sequence of updates. Using this data you should generate two output files:

1. 'twma_twstd_intervals.csv' should include the TWMA and TWSTD values at 1-second intervals, starting 30 seconds after the first input data point.
2. 'twma_twstd_timestamps.csv' should include the TWMA and TWSTD values for each timestamp in the input file 'security1.csv' (skipping the timestamps in the first 30 seconds of the data).

Part 2: Calculation of Time-Weighted Moving Median (TWMM)

Definition:

$TWMM_t$ is the timed-median of the window frame that ends at time 't'. Specifically, $TWMM_t = m$ if and only if the price in the window frame was above or equal to 'm' for 15 seconds and below or equal to 'm' for 15 seconds.

Task:

Add the method GetCurrentTWMM(t) to your implementation from Part 1. The method will return $TWMM_t$ as defined above.

As in Part 1, you should generate the two output files:

1. 'twmm_intervals.csv' with TWMM values at 1-second intervals, starting 30 seconds from the first timestamp.
2. 'twmm_timestamps.csv' with TWMM values for each timestamp in the input file 'security1.csv' (skipping the timestamps in the first 30 seconds of the data).

Performance Expectations:

Your implementations should focus on correctness as well as on efficiency of each of the methods. It should scale well if the window size and/or the density of the input data points is increased.

Please submit your code along with the requested output files.

Diagram 1:

Suppose we have the following timestamped prices:

P1, 12:01:02.725000000

P2, 12:01:11.800000000

P3, 12:01:27.300000000

P4, 12:01:33.150000000

If, for example, we look at the window frame ending at 12:01:44.25 (starting at 12:01:14.25) it will include P2,P3,P4 (but not P1 which ends before the start of the window.). The duration of P2 in the window will be 13.05 seconds (only from the start of the window at 12:01:14.25 and not from when it actually first appeared earlier). The duration of P4 is 11.1 seconds (from when it appeared until the end of the window frame.)

